

# Software Architecture of a System Designed for Churn Analysis and Customer Retention in Banking Industry

Erdal Bozan<sup>1</sup>, Guven Gul<sup>1</sup>, Umit Tigrak<sup>1</sup>, Emir Demirci<sup>1</sup>, Hakan Saribiyik<sup>1</sup>, Mehmet S. Aktas<sup>2</sup>

<sup>1</sup>R&D Center, Fibabanka, Istanbul, Turkey

<sup>2</sup>Yildiz Technical University, Istanbul, Turkey

{erdal.bozan, guven.gul, umit.tigrak, emir.demirci, hakan.saribiyik}@fibabanka.com.tr, [aktas@yildiz.edu.tr](mailto:aktas@yildiz.edu.tr)

**Abstract**— Within the scope of this research, a generic software architecture for the banking industry is proposed and studied to improve the analysis regarding customer retention and attrition. The prototype of the proposed architecture model is improved and its performance is tested in terms of scalability and responsiveness. The results are promising and indicating that the proposed system is serviceable.

**Keywords** — *customer attrition/churn and retention models, banking domain, software architecture.*

## I. INTRODUCTION

The increasing amount of data in-line with advances in technology and science has increased the interest in data science and machine learning techniques. Data science and machine learning techniques have been used as a tool to provide solutions for the needs of the companies in a wide range of industries. These techniques have also been used widely to analyze and improve customer satisfaction. In particular, considering the fact that the cost of attaining new customers is far higher compared to customer retention, the importance of the analysis of customer attrition for companies comes in sight.

Within the scope of this study, generic software architecture of data analysis platform is proposed particularly for companies in the banking industry as a response for the following needs: customer recognition, appropriate product-customer matching, optimization of product recommendations based on maximizing revenue and customer satisfaction, analyzing customer attrition behaviors, recommending best possible actions in the proper order for customer retention.

First prototype of this architecture has been developed in order to demonstrate the usability of the proposed software architecture. The performance tests have been conducted on the application as to the scalability and responsiveness. The successful results have been achieved and have demonstrated the usefulness of the proposed software architecture.

The organization of this paper is as follows: In Part 2 of the study, the definition of the problem, which has been studied currently, is indicated within the scope of this study. In Part 3, the literature survey concerning the study is described and the proposed software architecture is introduced as part of the study in Part 4. In Part 5, a performance evaluation carried out on the prototype and its results are demonstrated. In Part 6, the experiences as a result of the study are summarized and its future studies are discussed.

## II. PROBLEM DESCRIPTION

The number of customers can reach the millions in the banking industry. In addition to a wide range of financial services (products) offered for customers in banking, the

campaigns conducted for these products are based on expert opinion.

The high product diversity and number of customers requires a system that can make the right product suggestions for the right customer. Therefore, improving the approach based on expert opinion has become the primary target in the banking industry by using prediction techniques based on machine learning and optimization techniques. In this context, machine learning techniques are required to understand customer needs and behaviors, which creates opportunities in up-sells and cross-sells. It is also an important requirement that transforming decisions for product-customer matching by using optimization of the outputs of machine learning models into a single decision, which will create the best value for both customers and the company.

It is an important requirement that the number of predictive models required by such a system can be high and that the success of the models is analyzed automatically, and the performance of the models is monitored. The need for a software architecture system that will provide customer tendency models -including probability of purchasing/actively using specific banking products- is clear.

Customer attrition analysis is a type of analysis that is used for predicting the probability of losing existing customers. This kind of analysis gives opportunities to customer relations management for finding solutions to proactively prevent customer losses. Customer loss analysis is a study aiming to predict the probabilities of active customers to terminate their relations with the company. The data that belongs to the customers, who in the past terminated their relations with the company, is used as a training data set. This kind of problem is handled under the category of supervised learning. With limited sources, many campaigns are conducted for customer retention. However; as it is not possible to carry out these campaigns for all the customers, it is necessary to predict in advance the customers that have the tendency to terminate their products/relations. Within the scope of this research, it is aimed to conduct the specific retention campaigns as a response to this requirement.

The recommendations based on these models should be optimized for both the revenue expectations of companies and customer satisfaction.

Apart from providing their customers with banking products that will turn into economic value, banks aim to ensure customer satisfaction and good relations with customers. For ensuring customer satisfaction, it is necessary to model customer needs and behaviors by monitoring customer bank transactions. These models as a result of it are used to analyze and predict possible

customer losses in advance. In customer attrition analysis, it is necessary to have such a software that will give opportunities to create automatic models for predictions, to manage them, and to monitor the model performances.

As a result of the customer attrition analysis, it is crucial to determine the best possible actions in the proper order for preventing customer attrition.

Another important requirement is to include newly developed models into the system and to manage newly added modules without any requirement of changing system codes.

In line with these necessities, we define the requirements of the software architecture system that can achieve customer retention and attrition regarding banking as follows:

(-) Profiling customers who have similar or different product use and banking relation history is a requirement. Customer profiles should be created based on spending habits and bank product use behaviors. As customers are different from each other in many aspects, grouping customers according to their behaviors enables opportunities to create an appropriate product-customer matching.

(-) It is important to provide necessary daily and streaming data into the system, as the system uses this data to train the machine learning models, to score them and optimization models. Another important point is that the system should provide prediction and optimization results whenever needed.

(-) Models need to be created and implemented easily. In this context, the user that will load the models into the system will need a user interface, and the performances of machine learning models that have been implemented in the system need to be measurable.

(-) It is also expected that the same system should be used for both training models/predictions and monitoring model performance. In this way, the need to easily compare the predictions produced over a single system with the actual results of these predictions can be addressed.

(-) The recommendations created by the models should be optimized in a way that it should fulfill both maximum company revenue and maximum customer satisfaction. The system should also enable user to monitor the performance of the optimization.

(-) The system should be able to continue its functionality even if it is independent of these technologies used at a lower level and the technologies used at the lower level change. In this way, the system will be flexible in operating with open source technologies, new software, and different libraries. This will ensure that the system has the flexibility to work with up-to-date libraries or new languages, even in the case of updating or changing the programming languages of open source libraries used to develop models.

### III. LITERATURE REVIEW

In the customer attrition analysis, logistic regression models [1], support vector machine [2], artificial neural networks [3], and XGBoost [4] techniques are used, respectively. Among probabilistic learning models, logistic regression technique is quite popular for supervised learning problems due to its simple

mathematical structure and its easiness for inference [5]. However, in this study, rather than the application of customer attrition analysis methods, a proposal is made for a software architecture where these models can be trained, applied, and their performance can be compared.

Today in different fields, we see that management of data and metadata is provided based on generic software architecture. In the study by Tas et al. [6]; hybrid and generic software architecture are recommended for the collection and management of provenance metadata. When it is examined, it is seen that facade design pattern is used in this software architecture. In this software architecture; by using web services, it is possible to add metadata into the system, to update and to query it. In the study by Baeth et al. [7], generic software architecture is proposed for a software that can detect in real-time whether the policies determined by the people regarding personal information privacy on social media are violated. This software architecture uses facade design pattern, as well. In this architectural structure, the subscription/publish based messaging infrastructure is used for the communication of modules with each other and with other applications. In the study by Aktas [8]; for cloud computing platforms, software architecture is recommended as a monitoring tool. While the recommended architecture can follow up the performances of the virtual machines that operate on different cloud computing platforms, it is independent of the technologies that are used by these platforms. This software architecture has been developed based on facade design pattern, too. Similar to the studies mentioned, the software architecture proposed in the scope of this research uses facade design patterns. However, in this study, generic software architecture is proposed in the field of banking, which enables the production and implementation of models that can make customer attrition analysis, produce product suggestions models for economic benefit, and manage models, produce and manage promotional models for customers.

It is seen that there are different research studies in the literature that explain how to manage data and metadata – data that describes data [8-10]. It is observed that researches are also carried out in the field of information systems for distributed data storage platforms and in the field of searching data in a distributed environment [11-14]. Unlike these studies, this study focuses on a hybrid generic software that produces, implements, and manages models for customer attrition and retention analysis and related actions.

### IV. PROPOSED METHODOLOGY

In this manuscript, we describe a design and implementation details of a software architecture to support attrition and retention predictions and product recommendation strategies in the banking industry. The proposed software architecture is design to manage both supervised and unsupervised machine learning models. Supervised models are used to conduct attrition analysis. The unsupervised models are used to recommend products and next best actions. The unsupervised models include association rules, sequential pattern rules, content-based recommendation models, and collaborative filtering based

recommendation models. Figure 1 depicts the abstract software layers of the proposed software architecture.

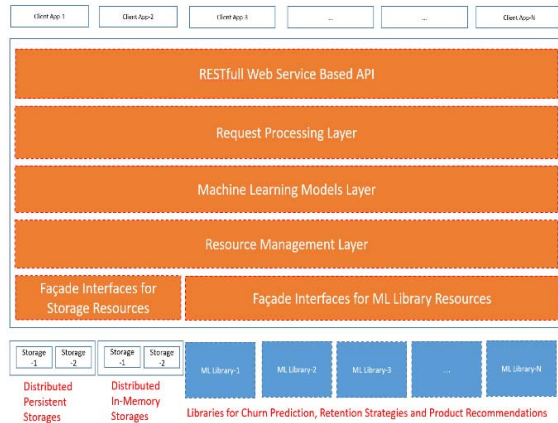


Figure 1 Abstract Software Layers for the Proposed Software Architecture

The proposed software architecture consists of various abstract software layers: RESTful Web Service based application programming interface, Request Processing Layer, Machine Learning Models Layer, Resource Management Layer, and Facade Interface to underlying subsystems. The framework is designed to provide a generic and scalable data processing architecture. The application programming interface layer provides a uniform programming interface to client application to manage machine learning models. The Request Processing layer is responsible for handling incoming requests. The models modles layer provides machine learning models in a way that the models can be queried in a low-latency way. The resource management layer is responsible for managing sub-systems. The proposed software architecture utilizes the Facade Design Pattern, which hides the complexities of the sub-systems. It provides an interface to the systems' inner-components from where these components can access to the sub-systems. Figure 2 gives a detailed picture of this framework architecture.

The proposed framework interacts with various data sources and action services. Data sources may include various types of banking transaction data set such as loan account transaction data, credit card account transaction data, and investment account transaction data. Data may be fed into the system as batch data. Action services my include notification services such as e-mail and sms notifications.

The framework has various components that include Hybrid Recommender Model Manager, Churn Prediction Model Manager, Rule Mining Model Manager, Optimization Module, Data Preprocessing Module, Model Training Management Module, Product/Service Recommendation Module, Churn Prediction Module, Next Best Action Prediction Module, Monitoring, and Reporting Module.

The framework will also include various programming interfaces which include a)Product Recommendation API, b)Next-Best-Action Recommendation API, c)Churn Analysis API, d)Data Pre-Preprocessing Management API, e)Model Management API and f) Monitoring and Reporting API.

1)Semantics for Data Representation of the Models: The proposed framework will support an industry standard for the semantics of the model representation. It will use an XML-based standard, the Predictive Model Markup Language (PMML). PMML provides a way to describe and exchange produced by machine learning algorithms.

2)Semantics for Product Recommendation API: The proposed architecture will provide a RESTful web service based programming interface to recommend products/services to the client applications. Given the banking customer-id, the system will be able to provide a list of ranked recommendations. This programming interface will provide different recommendation functions for varying recommender models such as association-rule based recommender model, content-based recommender model, collaborative-filtering based recommender model. The programming interface will also provide a function that can provide a recommendation which is based on an optimization function where the goals are maximization of both company profits and customer satisfaction.

3)Semantics for Next-Best Action Recommendation API: The system will utilize the RESTful web service based API to recommend next best action to be taken as part of a retention strategy. Here, given the customer-id, the system will provide a list of sequenced actions.

4)Semantics for Churn Analysis API: The system will utilize RESTful web service based programming interface to provide functions to conduct churn analysis. These functions will include churn prediction function for given customer-id(s) and a selected model. There will also be functions that can utilize more than one model to conduct a churn prediction. The programming interface will provide reporting functions that will give the list of existing supervised learning models that are in use.

5)Data Pre-Preprocessing Management API: The system will provide RESTful programming interface to provide data-preprocessing steps to enable an ML-ready dataset. Here, the API will include functions for normalization, outlier detection and extraction, missing data imputation, feature selection, feature dimension reduction. The client applications will be able to use this interface and convert the raw dataset to a ML-ready dataset.

6)Model Management API: The system will provide a RESTful programming interface to enable the management of ML models. The API will provide various functions such as training a model, serializing and deserializing a model, activating, and deactivating a model. The API will also include functions that report the list and status of existing models.

7)Monitoring and Reporting API: The system will provide a RESTful web service programming interface that will include functions providing analytics information on how the models are used. The analytics information will include various metadata o

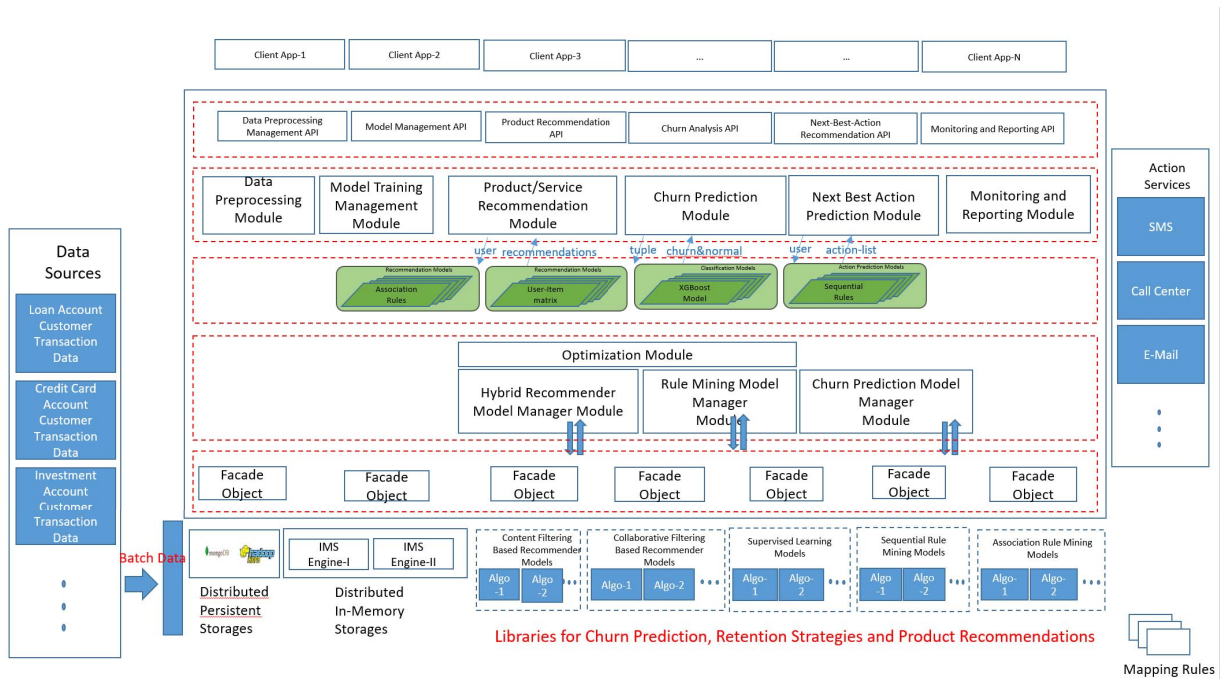


Figure 2 Proposed Software Architecture

models such as the count of usage, the success rate, creation time, size of the training data, etc.

Figure 2 illustrates how the client applications will interact with the system by utilizing the APIs such as a)Data Pre-Preprocessing Management API, b)Model Management API c)Product Recommendation API, d)Churn Analysis API, e)Next-Best-Action Recommendation API, and f) Monitoring and Reporting API.

The user will utilize the Data Preprocessing Management API to convert raw banking transaction datasets into ML-ready datasets. This request will be handled by the Data Collection and Preprocessing Module. The user will be able to utilize varying data preprocessing functions such as normalization, outlier detection and extraction, missing data imputation. Here, the user will utilize the Model Management API to make a request for ML model creation and training. This request will be handled within the serving layer by the Model Training Management Module. The user will be able to select one of the available ML-ready dataset and choose the ML algorithm of their choice and create a job for Model construction&training. Once an ML construction job is created, the job will be handled by one of the Model Manager Modules: Hybrid Recommender Manager Module, Churn Prediction Manager Module, and Rule Mining Manager Module. Each Manager Module utilizes facade design pattern to interact with underlying ML libraries. The mapping between the Manager Modules and underlying subsystems are configured in Mapping Files. Hence, if there is a need for changing the underlying ML library, this only requires an update in the Mapping XML files. On one hand, Hybrid Recommender Module is responsible for producing content-filtering and collaborative filtering based recommender models (i.e. user-

product rating matrices). On the other hand, Rule Mining Manager Module produces association rule-based recommendation models. Here, we use an Optimization Module, which is responsible for optimizing the recommendation strategy in a way that it should fulfill both maximum company revenue and maximum customer satisfaction. The user will interact with the Product Recommendation API to retrieve the list of recommendations for a given customer-id. The users will have the ability to select the recommendation model from which they want to receive their recommendations. Here, these requests will be handled by the Product/Service Recommendations Module, which will utilize one of the existing recommendation models in the system. The user will interact with the Churn Analysis API in order to ask for churn predictions. To address these requests, this API utilizes the Churn Prediction Module that in turn uses one of the pre-trained supervised learning models. The user has the ability to choose the supervised model to produce the predictions. This API also has functions than would return the success rates of the existing models. The user will interact with the Next-Best-Action Recommendation API to retrieve a list of ordered actions that need to be taken as part of retention strategy. This API uses Next Best Action Recommendation Module, which in turn utilizes a pre-generated model (i.e. sequential rules) to predict the next best action list. Finally, the user interacts with the Monitoring and Reporting API to request metadata on existing models in the system. This API utilizes the Monitoring and Reporting Module to retrieve information how the models have produced successful predictions.

## V. EVALUATION

We performed an evaluation test on the prototype of the proposed software architecture. In particular, we investigated how well the system performs in creating and training the supervised models with and without the use of facade design pattern. To do this, we evaluated the system performance under increasing dataset sizes. The system was tested with two subsystems (i.e. Xgboost and Lightgbm libraries).

The system was tested using a Jupyter Lab environment equipped with 72 core and 64GB RAM running on Linux Redhat operating system. We obtained 30 observations for each test and recorded average and standard times for latencies. Figure 1 illustrates the result of the first test where we used Xgboost library as the subsystem, while Figure 2 illustrates the results of the second test where we used Lightgbm library as the subsystem.

The test results indicate that the prototype implementation of the proposed software architecture achieves negligible processing overhead. The results showed that the use of facade design pattern only increases the system performance in the order of microseconds. The results also showed that the processing overhead does not change with increasing data sizes.

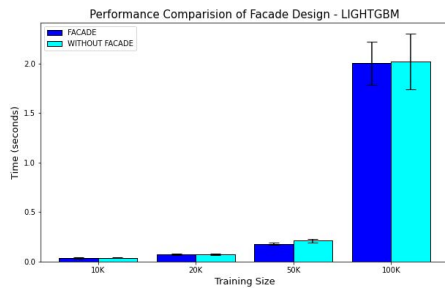


Figure 3 The results of the first performance test where the system is tested using a LightGBM as the subsystem.

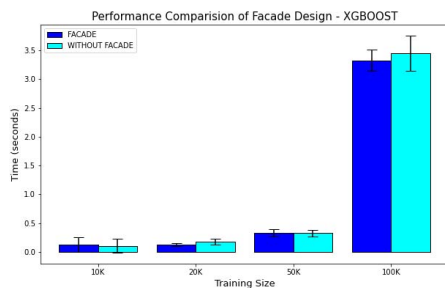


Figure 4 The results of the second performance test where the system is tested using XGBoost as the subsystem library.

## VI. CONCLUSION AND FUTURE WORK

In this study, we introduced a software architecture for model management software. The software was designed based on the requirements that were identified in the banking industry. The proposed software architecture utilizes facade design pattern. The facade design pattern is used in order to hide the complexities of subsystems.

We tested the system to find out whether the system is able to work with more than one subsystem without requiring code-change the prototype system. We also tested the system from the perspective of performance and measure the latency with/out facade design pattern usage. The test results indicate that the prototype implementation of the proposed software architecture achieves negligible processing overhead. The results also showed that the processing overhead does not change with increasing data sizes.

In the future work, we plan on investigating the system performance for other types of subsystems such as content and collaborative filtering subsystems, association and sequential rule mining subsystems. We also plan on investigating how the system performance in responding the client's requests for churn analysis and product recommendations.

## ACKNOWLEDGEMENT

This study was supported by Fibabanka Research and Development Center. The authors thank Fibabanka, which is a corporate banking company in Turkey, for enabling this study to take place.

## REFERENCES

- [1] Lemmens, Aurélie & Gupta, Sunil. (2019). Managing Churn to Maximize Profits. SSRN Electronic Journal. 10.2139/ssrn.2964906.
- [2] Hossain, Md Mosharaf & Miah, Mohammad. (2015). Evaluation of different SVM kernels for predicting customer churn. 1-4. 10.1109/ICCITech.2015.7488032.
- [3] Khan, Yasser & Shafiq, Shahryar & Naeem, Abid & Ahmed, Sheeraz & Safwan, Nadeem & Hussain, Sabir. (2019). Customers Churn Prediction using Artificial Neural Networks (ANN) in Telecom Industry. International Journal of Advanced Computer Science and Applications. 10. 10.14569/IJACSA.2019.0100918.
- [4] Ahmad, A.K., Jafar, A. & Aljoumaa, K. J Big Data (2019) 6: 28. <https://doi.org/10.1186/s40537-019-0191-6>
- [5] Duda, Richard & Hart, Peter & G.Stork, David. (2001). Pattern Classification.
- [6] Tas, Y., Baeth M.J., Aktas, M.S., (2016) An Approach to Standalone Provenance Systems for Big Provenance Data, The International Conference on Semantics, Knowledge and Grids on Big Data (SKG-16).
- [7] Baeth, M.J., Aktas, M.S., (2018) . An approach to custom privacy violation detection problems using big social provenance data, Concurrency and Computation-Practice & Experience, Vol: 30, Issue:21.
- [8] Aktas, M.S., (2018) Hybrid cloud computing monitoring software architecture, Concurrency and Computation: Practice and Experience, Vol: 30, Issue:21.
- [9] Baeth, M.J., Aktas, M.S., (2019). Detecting misinformation in social networks using provenance data, Concurrency and Computation-Practice & Experience, Vol: 31, Issue:3.
- [10] Riveni, M. et al. (2019). Application of provenance in social computing: A case study, Concurrency and Computation-Practice & Experience, Vol: 31, Issue:3.
- [11] Aktas, M.S. et al. (2004). A web based conversational case-based recommender system for ontology aided metadata discovery, The 5th IEEE/ACM International Workshop on Grid Computing, pp:69-75.
- [12] Aktas, M.S. et al, (2007). Fault tolerant high-performance Information Services for dynamic collections of Grid and Web services, Future Generation Computer Systems, Vol:23, Issue: 3.

- [13] Pierce, M.E. et al. (2008). The QuakeSim project: Web services for managing geophysical data and applications, *Pure and Applied Geophysics*, Vol:165, Issue: 3-4, pp. 635-651.
- [14] Aydin, G. et al. (2005). SERVOnet complexity computational environments (CCE) integrated performance analysis, *The 6th IEEE/ACM International Workshop on Grid Computing*.